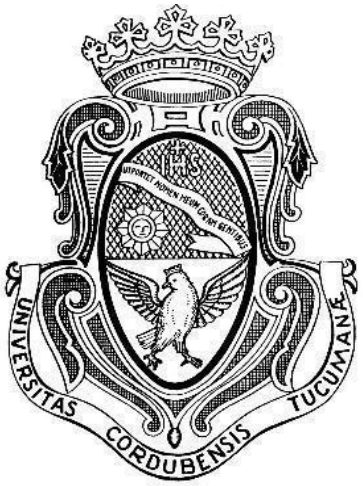


UNIVERSIDAD NACIONAL DE CÓRDOBA

FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES

COMUNICACIONES DE DATOS

INFORME DE LABORATORIO 5

GRUPO
LA LA LAN

ALUMNOS

Castellano, Juana - 45.408.569

Fernandez Bonilla, Pilar - 44.475.509

Palacio Luis- 40609862

Rodriguez Cappellari Leandro -44295349

Consignas:

Al final del TP tienen que poder contestar esa pregunta con detalle, y haber visto con Wireshark buena parte de la respuesta. Vamos a ir de lo más simple (un ping) a lo más propio (un programa escrito por ustedes), y de una sola computadora a dos.

Antes de empezar:

- Capturen solamente tráfico propio, en sus computadoras o en el router/hotspot del grupo. No analicen el tráfico de otras personas.
- Para capturar, Wireshark necesita permisos: en Windows alcanza con haber instalado Npcap; en Linux, ejecutar Wireshark con sudo o agregar su usuario al grupo wireshark (según la distribución).
- En Windows usen python (o py); en Linux, python3. En los ejemplos escribimos python.
- Guarden cada captura importante: forman parte de la entrega.
- Todos los filtros de este TP para wireshark son display filters (se escriben en la barra arriba de la lista de paquetes). No los confundan con los capture filters (que se configuran antes de capturar y usan otra sintaxis). Si el filtro está bien escrito la barra se pone verde; si está mal, roja.

Puertos que vamos a usar en todo el TP:

Protocolo	Puerto	Dónde se usa
TCP	12000	ncat (TCP), tcp_server.py, tcp_client.py, filtro tcp.port == 12000
UDP	12001	ncat -u (UDP), udp_server.py, udp_client.py, filtro udp.port == 12001

Usamos puertos distintos para que los filtros de TCP y de UDP no se mezclen al comparar capturas. (En el punto 4 van a pensar si realmente hacía falta que fueran distintos.)

1) ICMP y primer contacto con Wireshark.

Empezamos con la herramienta de red más conocida: ping. La vamos a usar como "generador de tráfico" para aprender a leer una captura. Investigar brevemente y documentar en el trabajo:

a) ¿Qué es ICMP y para qué se usa? ¿Transporta datos de aplicaciones como lo hacen TCP o UDP?

ICMP (Internet Control Message Protocol, RFC 792) es un protocolo de control y diagnóstico de la capa de red. Los equipos lo usan para avisar errores y estados de la red. **No transporta datos de aplicaciones** como TCP o UDP. No tiene puertos ni conexiones, y no entrega datos a un proceso. Sus mensajes son información sobre la propia red.

b) ¿Qué relación tiene con IP? ¿Viaja dentro de IP, al lado de IP o debajo de IP? ¿Cómo sabe el receptor que el contenido de un paquete IP es ICMP?

ICMP viaja **dentro de IP**: el mensaje ICMP va en el payload de un datagrama IPv4. Aun así, se lo considera parte de la capa de red, porque es un protocolo auxiliar de IP y toda implementación de IP debe soportarlo. El receptor sabe que el contenido es ICMP por el campo **Protocol del encabezado IPv4 = 1**. Como comparación, TCP es 6 y UDP es 17.

c) ¿Qué hace ping? ¿Qué son un Echo Request y un Echo Reply? ¿Qué campos de ICMP permiten distinguirlos?

Ping verifica si un host es alcanzable y mide el tiempo de ida y vuelta (RTT). Para eso envía un **Echo Request** y espera el **Echo Reply**. El destino debe devolver el Reply con el mismo identificador, número de secuencia y datos. Los dos mensajes se distinguen por los campos **Type y Code**:

Echo Request: Type = 8, Code = 0

Echo Reply: Type = 0, Code = 0

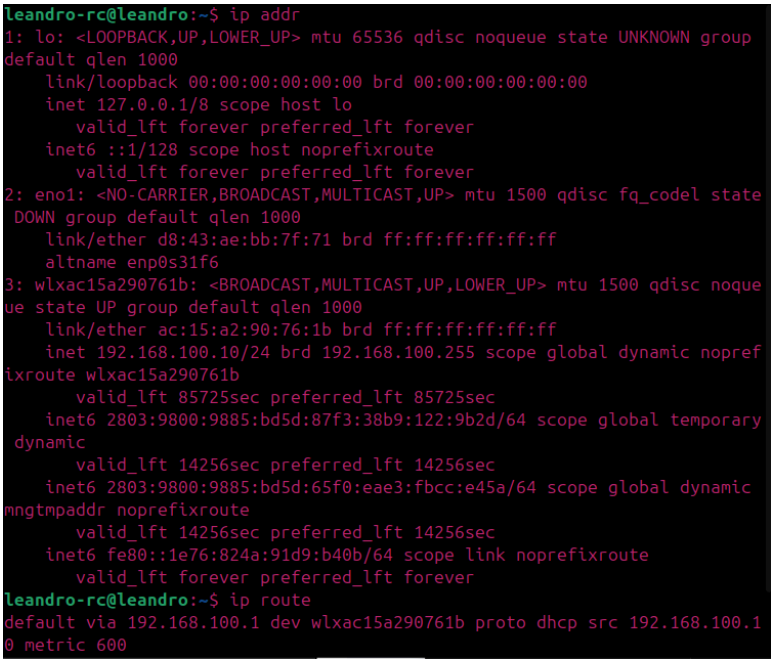
d) ¿Qué información mínima contiene un mensaje ICMP de tipo Echo?

Un mensaje ICMP Echo tiene un encabezado de 8 bytes seguido de los datos. El campo Type indica si es un pedido (8) o una respuesta (0), y el Code vale 0. El checksum se usa para detectar errores. El identificador y el número de secuencia permiten saber a qué ping corresponde cada respuesta. Por último van los datos, que la respuesta devuelve iguales.

Averiguar la configuración de red de alguna de las computadoras del grupo: dirección IPv4, máscara, gateway por defecto y dirección MAC de la interfaz que usan (Wi-Fi o cableada).

```
Windows: ipconfig /all
Linux: ip addr y ip route
```

La configuración de red se obtuvo con los comandos ip addr e ip route. La computadora se conecta por Wi-Fi a través de la interfaz wlxac15a290761b, con dirección IPv4 192.168.100.10 y máscara 255.255.255.0 (/24). El gateway por defecto es 192.168.100.1 y la dirección MAC de la interfaz es ac:15:a2:90:76:1b.



Abrir Wireshark, elegir la interfaz por la que salen a Internet (Wi-Fi o Ethernet; la que muestra actividad en el gráfico) y comenzar la captura. En una terminal, hacer ping a la IP del gateway por defecto y luego a una IP de Internet:

```
# Windows (envía 4 pings por defecto)
ping -4 <IP_del_gateway>
ping -4 8.8.8.8

# Linux (sin -c no se detiene; cortar con Ctrl+C)
```

```
ping -4 -c 4 <IP_del_gateway>
ping -4 -c 4 8.8.8.8
```

```
leandro-rc@leandro:~$ ping -4 -c 4 192.168.100.1
PING 192.168.100.1 (192.168.100.1) 56(84) bytes of data.
64 bytes from 192.168.100.1: icmp_seq=1 ttl=64 time=111 ms
64 bytes from 192.168.100.1: icmp_seq=2 ttl=64 time=2.59 ms
64 bytes from 192.168.100.1: icmp_seq=3 ttl=64 time=2.70 ms
64 bytes from 192.168.100.1: icmp_seq=4 ttl=64 time=1.72 ms

--- 192.168.100.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 1.715/29.404/110.616/46.889 ms
leandro-rc@leandro:~$ ping -4 -c 4 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=118 time=18.2 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=118 time=17.7 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=118 time=18.9 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=118 time=90.4 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 17.687/36.303/90.408/31.240 ms
```

El Echo Request enviado tiene TTL = 64, valor por defecto en Linux. El Echo Reply de 8.8.8.8 llegó con TTL = 118, mientras que el del gateway llegó con TTL = 64. La diferencia se debe a que cada sistema elige su propio TTL inicial y cada router que atraviesa el paquete lo decrementa en 1. Suponiendo un TTL inicial de 128 en el servidor de Google, la respuesta atravesó aproximadamente 10 routers. El gateway, al estar en la misma red, responde sin saltos intermedios.

Usamos -4 y direcciones IP (no nombres) para forzar IPv4. Si hacen ping a un nombre, puede usarse IPv6 y el tráfico aparece como icmpv6, que no coincide con el filtro icmp.

Detener la captura y aplicar el filtro:

```
icmp
```

No.	Time	Source	Destination	Protocol	Length	Info
3065	7.741772692	192.168.100.1	192.168.100.10	ICMP	98	Echo (ping) request
3066	7.741785982	192.168.100.10	192.168.100.1	ICMP	98	Echo (ping) reply
3067	7.743489782	192.168.100.1	192.168.100.10	ICMP	98	Echo (ping) request
3068	7.743494702	192.168.100.10	192.168.100.1	ICMP	98	Echo (ping) reply
3147	18.586381383	192.168.100.1	192.168.100.10	ICMP	98	Echo (ping) request
3148	18.586392254	192.168.100.10	192.168.100.1	ICMP	98	Echo (ping) reply
6801	24.185391902	192.168.100.10	192.168.100.1	ICMP	98	Echo (ping) request
6802	24.295998547	192.168.100.1	192.168.100.10	ICMP	98	Echo (ping) reply
6804	25.187120660	192.168.100.10	192.168.100.1	ICMP	98	Echo (ping) request
6805	25.189692502	192.168.100.1	192.168.100.10	ICMP	98	Echo (ping) reply
6808	26.188022692	192.168.100.10	192.168.100.1	ICMP	98	Echo (ping) request
6809	26.190709678	192.168.100.1	192.168.100.10	ICMP	98	Echo (ping) reply
6811	27.189085118	192.168.100.10	192.168.100.1	ICMP	98	Echo (ping) request
6812	27.190782965	192.168.100.1	192.168.100.10	ICMP	98	Echo (ping) reply
7348	37.722068252	192.168.100.10	8.8.8.8	ICMP	98	Echo (ping) request
7350	37.740268033	8.8.8.8	192.168.100.10	ICMP	98	Echo (ping) reply
7353	38.256987724	192.168.100.1	192.168.100.10	ICMP	98	Echo (ping) request
7354	38.256998081	192.168.100.10	192.168.100.1	ICMP	98	Echo (ping) reply
7357	38.258797665	192.168.100.1	192.168.100.10	ICMP	98	Echo (ping) request
7358	38.258805131	192.168.100.10	192.168.100.1	ICMP	98	Echo (ping) reply
7370	38.723391692	192.168.100.10	8.8.8.8	ICMP	98	Echo (ping) request
7372	38.741071458	8.8.8.8	192.168.100.10	ICMP	98	Echo (ping) reply
7382	39.725006113	192.168.100.10	8.8.8.8	ICMP	98	Echo (ping) request
7383	39.743910332	8.8.8.8	192.168.100.10	ICMP	98	Echo (ping) reply
7386	40.726009029	192.168.100.10	8.8.8.8	ICMP	98	Echo (ping) request

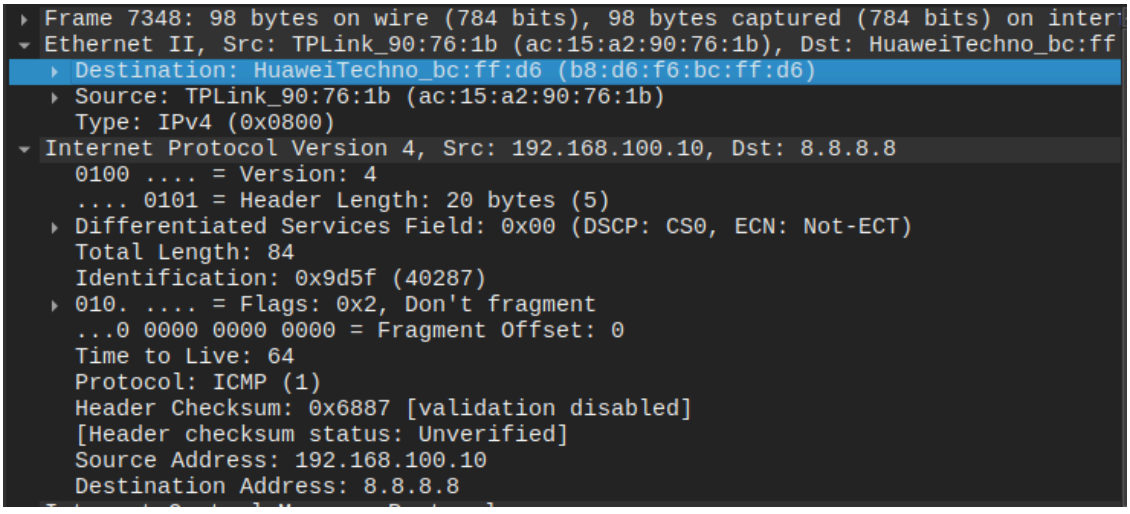
Al aplicar el filtro icmp quedan solo los mensajes de ping. Se observan los Echo Request enviados al gateway (192.168.100.1) y a 8.8.8.8, cada uno seguido de su Echo Reply. Todas las tramas miden 98 bytes.
En la captura también aparecen Echo Request enviados por el router (192.168.100.1) hacia nuestra computadora, que esta responde automáticamente. No fueron generados por nosotros: el router los usa para verificar qué equipos siguen conectados a la red.

Si la red no responde a pings hacia Internet, no es un error de ustedes: algunas redes filtran ICMP (aunque es raro, con datos del celu no debería pasarles). En ese caso trabajen con el ping al gateway.

Seleccionar un Echo Request y desplegar el panel de detalles (el del medio). Identificar las capas que muestra Wireshark y completar una tabla como esta:

Capa (como la nombra Wireshark)	Dirección/identificador origen	Dirección/identificador destino	¿Qué campo indica qué protocolo viene "adentro"?
Ethernet II	TPLink_90:76:1b (ac:15:a2:90:76:1b	HuaweiTechno_bc:ff:d6 (b8:d6:f6:bc:ff:d6)	Type= 0x0800(IPV4)
Internet Protocol Version 4	192.169.100.10	8.8.8.8	Protocol: ICMP (1)
Internet Control Message Protocol	-	-	Type = 8 (Echho(ping) request)
Datos / payload	-	-	Ninguno: es el último nivel, no hay otro protocolo dentro. Son 56 bytes de datos de ping

Ayuda: al hacer clic en una línea del panel de detalles, Wireshark resalta en el panel inferior (hexadecimal) los bytes que le corresponden. Úsenlo para "ver" dónde empieza y termina cada encabezado.
Responder, justificando con la captura (una captura de pantalla con el campo resaltado vale más que el valor copiado):



```
▾ Internet Control Message Protocol
    Type: 8 (Echo (ping) request)
    Code: 0
    Checksum: 0x2b51 [correct]
    [Checksum Status: Good]
    Identifier (BE): 8112 (0x1fb0)
    Identifier (LE): 45087 (0xb01f)
    Sequence Number (BE): 1 (0x0001)
    Sequence Number (LE): 256 (0x0100)
    [Response frame: 7350]
    Timestamp from icmp data: Oct 28, 2026 14:51:00.297475000 -03
    [Timestamp from icmp data (relative): 0.000008006 seconds]
  ▾ Data (40 bytes)
      Data: 101112131415161718191a1b1c1d1e1f202122232425262728292a2b2c2d2e2f3031
      [Length: 40]
```

a) La MAC destino del Echo Request enviado a 8.8.8.8, ¿es la MAC de 8.8.8.8? ¿De qué equipo es? Compárenla con la MAC destino del ping al gateway. ¿Qué conclusión sacan sobre el alcance de una dirección MAC frente al de una dirección IP?

La MAC destino del Echo Request enviado a 8.8.8.8 es b8:d6:f6:bc:ff:d6, que no pertenece a 8.8.8.8 sino al router de nuestra red (gateway 192.168.100.1, fabricante Huawei). Es la misma MAC destino que tiene el ping al gateway. Como 8.8.8.8 está fuera de nuestra red, la computadora le entrega la trama al router para que la reenvíe hacia Internet. La conclusión es que la dirección MAC solo tiene alcance dentro de la red local (un salto), mientras que la dirección IP identifica al destino final de punta a punta y no cambia en el camino.

b) Comparen un Echo Request con su Echo Reply (Wireshark los vincula en el campo [Response frame: …]). Hagan una lista de los campos que cambian y de los que se mantienen en Ethernet, IP e ICMP. ¿Por qué tiene sentido cada cambio? ¿Por qué el identificador y el número de secuencia se mantienen?

Capa	Que cambia?	Que se mantiene?
Ethernet	se invierte la MAC de origen y destino	Type 0x0800(IPV4)
IP	IP de origen y destino invertidas, TTL(64 → 118), identification, header checksum	version, largo del encabezado,largo total y protocolo
ICMP	type(8→0), checksum	code, identifier(BE y LE), sequence number, Data

Las direcciones se invierten porque ahora el que responde es el otro extremo. El TTL cambia porque lo pone otro equipo y cada router lo va bajando. El Type pasa de 8 a 0 porque el mensaje pasa de pedido a respuesta. Los checksums se recalculan porque cambió el contenido. El identificador y el número de secuencia se mantienen porque así ping sabe a qué pedido corresponde cada respuesta, y con eso calcula el tiempo de ida y vuelta y detecta si se perdió alguno.

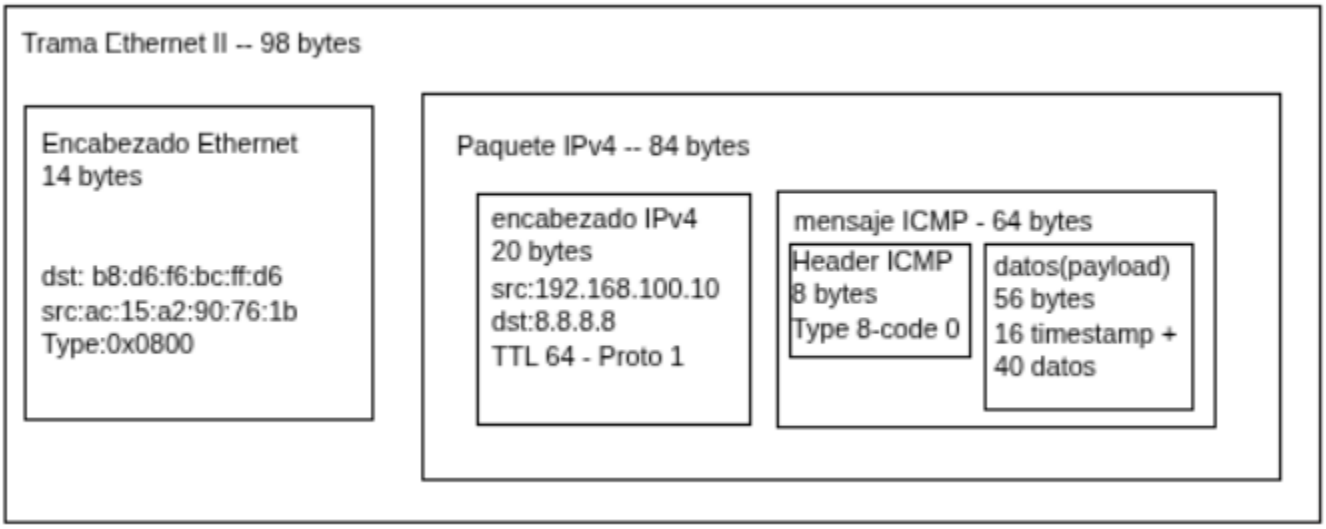
c) ¿Dónde está el payload de ping? ¿Cuántos bytes tiene y qué contiene? ¿Es igual en el Reply? Si en el grupo hay una computadora con Windows y otra con Linux, compárenlos: ¿qué les sugiere que sean distintos?

El payload está después del encabezado ICMP. En Linux tiene 56 bytes: los primeros 16 son una marca de tiempo con la hora de envío, que ping usa para calcular el tiempo de respuesta, y los 40 restantes son una secuencia de bytes de 0x10 a 0x37. El Echo Reply devuelve exactamente los mismos datos.
En Windows, ping manda 32 bytes con letras del abecedario (“abcdefghijklmnopqrstuvwabcdefghi”). Que sean distintos muestra que el contenido de los datos no lo define el protocolo ICMP, sino cada programa de ping; el protocolo solo exige que la respuesta devuelva lo mismo.

d) ¿Qué valor de TTL tiene el Echo Request que ustedes enviaron? ¿Y el Reply que llegó de 8.8.8.8? ¿Por qué no son iguales? (Pista: investiguen qué hace un router con el TTL.)

El Echo Request que enviamos tiene TTL = 64, que es el valor por defecto en Linux. El Reply de 8.8.8.8 llegó con TTL = 118. No son iguales porque cada equipo elige su propio TTL inicial y cada router que atraviesa el paquete le resta 1 (si llega a 0, lo descarta). Si el servidor de Google lo envía con 128, la respuesta pasó por unos 10 routers. En cambio, el Reply del gateway llegó con TTL = 64, porque está en nuestra misma red y no hay routers en el medio.

e) Dibujen la encapsulación del paquete que eligieron como "cajas dentro de cajas", indicando para cada caja qué tamaño en bytes tiene según Wireshark.



2) ARP: de una IP a una dirección MAC

En el punto anterior la trama Ethernet ya tenía una MAC destino. Pero ping solo conoce una IP. ¿De dónde salió esa MAC? Investigar brevemente:

a) ¿Qué problema resuelve ARP? ¿En qué capa lo ubicarían y por qué es discutible?

ARP (Address Resolution Protocol) permite obtener la dirección MAC correspondiente a una dirección IPv4 dentro de una red local. Esto es necesario porque, aunque las aplicaciones y la capa de red utiliza direcciones IP, las tramas Ethernet se entregan utilizando direcciones MAC.
Su ubicación en el modelo por capas es discutible,ya que trabaja con direcciones IP de capa 3, pero se transporta directamente sobre Ethernet y permite resolver direcciones MAC de capa 2. Se suele considerar un protocolo ubicado en la interfaz entre las capas de red y enlace.

b) ¿Qué es un ARP Request y un ARP Reply? ¿A quién se envía cada uno?

- *ARP Request:* mensaje que envía un host cuando conoce la IP destino, pero todavía no conoce la dirección MAC asociada a esa IP dentro de la red local. Esta se envía como broadcast (a todos los equipos de la LAN), debido a lo mencionado, que el emisor desconoce la MAC de destino.
- *ARP Reply:* respuesta del equipo que tiene esa IP, el mensaje informa su dirección MAC del host que hizo la consulta. Se envía unicast, directamente a la MAC del equipo que consultó.

c) ¿Qué es la caché ARP y por qué existe?

La caché ARP es una tabla temporal donde un host almacena asociaciones entre direcciones IPv4 y direcciones MAC que ya fueron aprendidas mediante ARP. Su objetivo es evitar realizar un nuevo ARP Request cada vez que se necesita enviar tráfico al mismo destino, reduciendo el tráfico broadcast y agilizando la comunicaciones en la red local. Es conveniente mencionar que, las entradas son temporales y pueden actualizarse o eliminarse con el tiempo.

d) Traten de responder con sus palabras: "Tengo la IP de una máquina de mi red local. ¿Cómo sé a qué dirección MAC debo enviarle la trama?"

Conociendo la IP de una máquina de mi red local, primero verifico si su dirección MAC ya está almacenada en la caché de la ARP. En caso de que no exista una entrada, mi equipo envía un ARP Request por broadcast preguntando: ¿Qué dispositivo posee esa IP?, la respuesta que viene de un ARP Reply indicando la dirección MAC de nuestro interés. A partir de ese momento, esa MAC puede utilizarse como destino de la trama Ethernet y su asociación queda almacenada temporalmente en la caché ARP.

e) Ver la caché ARP de su computadora y buscar la entrada del gateway: ¿La MAC asociada al gateway coincide con la MAC destino que vieron anteriormente?

Debido a que esta parte del TP se realizó desde una computadora y red diferentes a las utilizadas anteriormente, se repitió el envío de un ICMP Echo Request hacia 8.8.8.8 para realizar la comparación dentro del mismo entorno de red.

Mediante *ip route* se identificó al gateway:

```
juana@juana-Inspiron-15-3511:~$ ip route
default via 192.168.1.1 dev wlp1s0 proto dhcp src 192.168.1.76 metric 600
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
192.168.1.0/24 dev wlp1s0 proto kernel scope link src 192.168.1.76 metric 600
```

Luego, mediante *ip neigh show,* se comprobó la dirección MAC asociada a dicho gateway:

```
juana@juana-Inspiron-15-3511:~$ ip neigh show
192.168.1.1 dev wlp1s0 lladdr b4:3a:e2:8a:6e:98 REACHABLE
fe80::1 dev wlp1s0 lladdr b4:3a:e2:8a:6e:98 router STALE
```

Al analizar en Wireshark un Echo Request dirigido a 8.8.8.8, se observó que la dirección MAC destino del encabezado Ethernet también es b4:3a:e2:8a:6e:98.

Esto confirma lo observado en el punto anterior, cuando el destino IP se encuentra fuera de la red local, la computadora no envía la trama Ethernet directamente al host remoto, sino al gateway de la red. La IP identifica el destino final, mientras que la dirección MAC identifica al siguiente salto dentro de la red local.

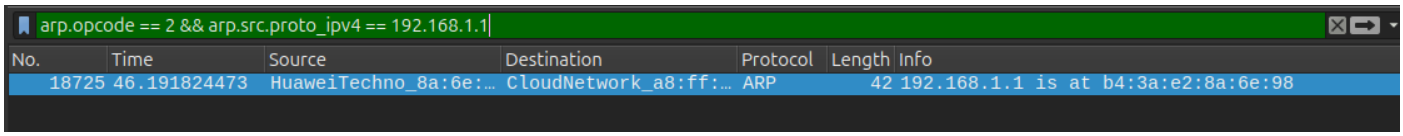
f) Generar tráfico ARP y capturarlo. Capturen en la interfaz Wi-Fi/Ethernet con el filtro arp. Como las entradas quedan guardadas en la caché, un ping a un equipo con el que ya hablaron puede no generar ningún ARP. Tienen varias formas de provocarlo (elijan la que puedan usar):
Optamos por la OPCIÓN B
Opción B (requiere administrador; el cambio es temporal): borrar solo la entrada del gateway y volver a hacerle ping. La entrada se vuelve a aprender sola en cuanto se necesita; no rompe la configuración de red.

ARP Request:

arp.opcode == 1 && arp.dst.proto_ipv4 == 192.168.1.1

No.	Time	Source	Destination	Protocol	Length	Info
2341	5.728470237	a8:6e:84:ea:7f:59	Broadcast	ARP	42	Who has 192.168.1.1? Tell 192.168.1.53
8583	17.813257821	a8:6e:84:ea:7f:59	Broadcast	ARP	42	Who has 192.168.1.1? Tell 192.168.1.53
15934	35.728529652	a8:6e:84:ea:7f:59	Broadcast	ARP	42	Who has 192.168.1.1? Tell 192.168.1.53
18713	46.134944645	CloudNetwork_a8:ff:…	Broadcast	ARP	42	Who has 192.168.1.1? Tell 192.168.1.76
19117	47.715502541	a8:6e:84:ea:7f:59	Broadcast	ARP	42	Who has 192.168.1.1? Tell 192.168.1.53
24066	65.731037679	a8:6e:84:ea:7f:59	Broadcast	ARP	42	Who has 192.168.1.1? Tell 192.168.1.53
28261	77.712303501	a8:6e:84:ea:7f:59	Broadcast	ARP	42	Who has 192.168.1.1? Tell 192.168.1.53
93566	216.702788566	a8:6e:84:ea:7f:59	Broadcast	ARP	42	Who has 192.168.1.1? Tell 192.168.1.53
98831	228.708261992	a8:6e:84:ea:7f:59	Broadcast	ARP	42	Who has 192.168.1.1? Tell 192.168.1.53
1025…	245.751656563	a8:6e:84:ea:7f:59	Broadcast	ARP	42	Who has 192.168.1.1? Tell 192.168.1.53
1053…	257.737393569	a8:6e:84:ea:7f:59	Broadcast	ARP	42	Who has 192.168.1.1? Tell 192.168.1.53

ARP Reply:



Analizar un ARP Request y su ARP Reply. Para cada uno completar una tabla como la siguiente:

Campo	ARP Request	ARP Reply
MAC destino (encabezado Ethernet)	ff:ff:ff:ff:ff:ff	10:6f:d9:a8:ff:bd
MAC origen (encabezado Ethernet)	10:6f:d9:a8:ff:bd	b4:3a:e2:8a:6e:98
Opcode	1(Request)	2 (Reply)
Sender MAC address	10:6f:d9:a8:ff:bd	b4:3a:e2:8a:6e:98
Sender IP address	192.168.1.76	192.168.1.1
Target MAC address	00:00:00:00:00:00	10:6f:d9:a8:ff:bd
Target IP address	192.168.1.1	192.168.1.76

Y responder:

a) **¿Por qué el Request va a una dirección broadcast y el Reply no? ¿Qué valor tiene Target MAC address en el Request y por qué?**

Como mencionamos en los puntos anteriores, el ARP Request se envía por broadcast porque el emisor conoce la dirección IP de destino, pero todavía desconoce la dirección MAC. Entonces, la trama Ethernet se dirige a *ff:ff:ff:ff:ff:ff,* alcanzando a todos los equipos de la LAN. En el Request, el campo *Target MAC Address* tiene el valor 00:00:00:00:00:00, ya que esa es la información que se intenta obtener. A su vez, el ARP Replay se envía de forma unicast porque el equipo que responde ya conoce la MAC del host solicitante a partir del Request recibido.

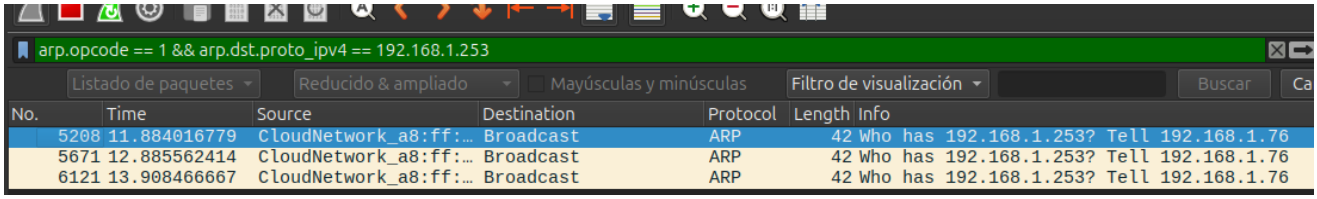
b) **En el encabezado Ethernet de la trama ARP, ¿qué valor tiene el campo Type? ¿Hay un encabezado IP? ¿Qué les dice eso sobre dónde "vive" ARP?**

En el encabezado Ethernet de la trama ARP el campo Type tiene del valor *0x0806,* que identifica al protocolo ARP.

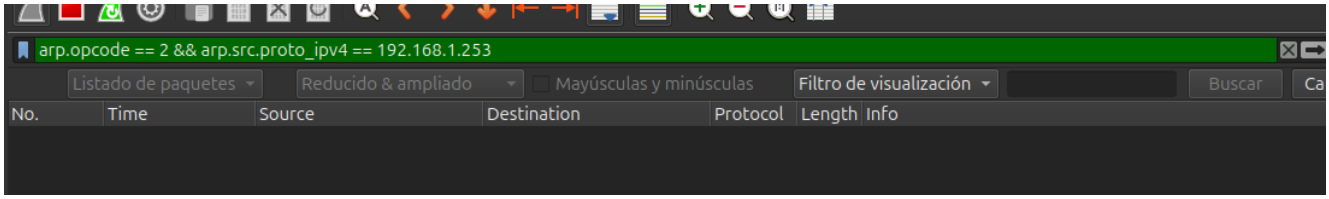
No existe un encabezado IP entre Ethernet y ARP, ya que ARP se encapsula directamente dentro de la trama Ethernet. Esto muestra que ARP funciona en la frontera entre la capa de enlace y la capa de red.

c) Con la Opción A (IP inexistente): ¿cuántos ARP Request aparecieron? ¿Hubo Reply? ¿Apareció algún ICMP Echo Request en la captura? Expliquen por qué.

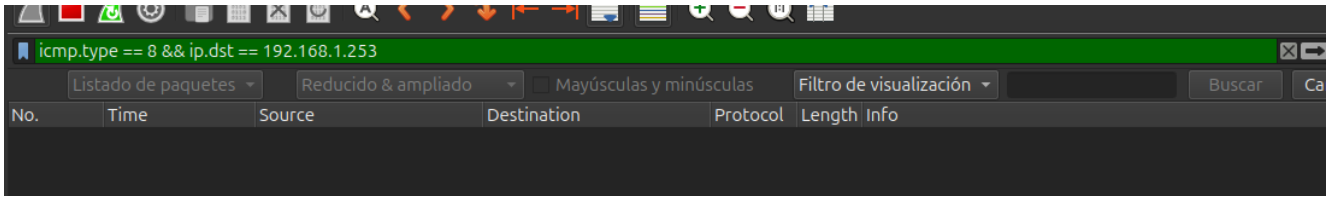
Siguiendo la opción A con IP inexistente observamos que aparecieron 3 ARP Request:



A la hora de queres ver si hubo Reply observamos que no aparece nada, por lo que afirmamos que no hubo



Y por último, tampoco observamos algún ICMP Echo Request:



Podemos concluir entonces que al hacer ping hacia una IP inexistente, 3 ARP Request intentaron resolver su dirección MAC.
No se recibió ningún ARP Reply, ya que ningún equipo respondió como propietario de esa dirección IP y tampoco se generó un ICMP Echo Request hacia el destino porque antes de poder enviar el paquete el host necesita conocer la dirección MAC correspondiente.

d) Volvieron a hacer ping al mismo destino un minuto después: ¿apareció ARP de nuevo? Revisen la caché ARP. ¿Qué ventaja tiene la caché y qué problema podría causar si una entrada quedará vieja?

En el primer intento de ping hacia una IP inexistente (observable en las imágenes del punto anterior) se observaron 3 ARP Request. Al repetir el ping aproximadamente un

minuto después, volvieron aparecer 3 ARP Request. Esto se debe a que ningún equipo respondió como propietario de dicha IP, por lo que no se pudo establecer la asociación IP-MAC válida para almacenar en la caché ARP. Como consecuencia, al intentar comunicarse nuevamente con ese destino, el sistema tuvo que repetir el proceso de resolución ARP.

La caché ARP tiene como ventaja que evita realizar consultas ARP repetidas cuando ya se conoce la relación entre una dirección IP y una dirección MAC, reduciendo tráfico broadcast y acelerando la comunicación. Sin embargo, si una entrada de la caché queda desactualizada, el host podría continuar enviando tramas a una dirección MAC incorrecta hasta que dicha entrada expire o sea actualizada.

3) TCP y UDP "a mano" con ncat

Hasta ahora observamos protocolos que usa el sistema operativo. Ahora vamos a comunicar dos procesos (dos programas) sin escribir código todavía. ncat es un programa que toma lo que escribimos en el teclado y lo manda por la red (y viceversa). La idea es empezar a generar tráfico cada vez más a bajo nivel (primero generamos tráfico con Packet Tracer, luego con la terminal del SO, ahora con un software a través de la CLI y finalmente ejecutaremos algunos scripts). Descargar netcat! para linux es bastante directo:

```
sudo apt install netcat-openbsd
```

Para windows, tienen la versión portable para descargar, es lo más cómodo pero seguramente tengan que apagar el defender un rato para bajarlo.

Investigar brevemente:

a) **¿Qué significa "establecer una conexión"? ¿Dónde "existe" una conexión TCP: en los cables, en los routers o en los extremos?**

Establecer una conexión TCP es sincronizar ambos extremos mediante SYN, SYN-ACK y ACK. La conexión existe como estado en los sistemas operativos de los extremos: secuencias, buffers y ventanas.

b) **¿Qué es un puerto? ¿Qué identifica el par (IP, puerto)?**

Un puerto es un número de 16 bits que permite al sistema operativo entregar los datos al socket correspondiente. El par (IP, puerto) identifica un extremo de comunicación dentro de un protocolo de transporte; TCP y UDP tienen espacios de puertos independientes.

c) **¿Qué significa que un proceso esté "escuchando" en un puerto?**

Significa que tiene un socket asociado a una dirección y un puerto, preparado para recibir comunicaciones. En TCP espera solicitudes de conexión (listen). En UDP espera datagramas, sin establecer una conexión.

Importante: capturar en loopback. En esta parte cliente y servidor corren en la misma computadora y se comunican por 127.0.0.1 o localhost. Ese tráfico no sale por la placa Wi-Fi: en Wireshark hay que elegir la interfaz de loopback:

Windows (Npcap): Adapter for loopback traffic capture. Linux: lo.

Van a notar que, según el sistema operativo, la captura en loopback no muestra un encabezado Ethernet o lo muestra con MAC 00:00:00:00:00:00.

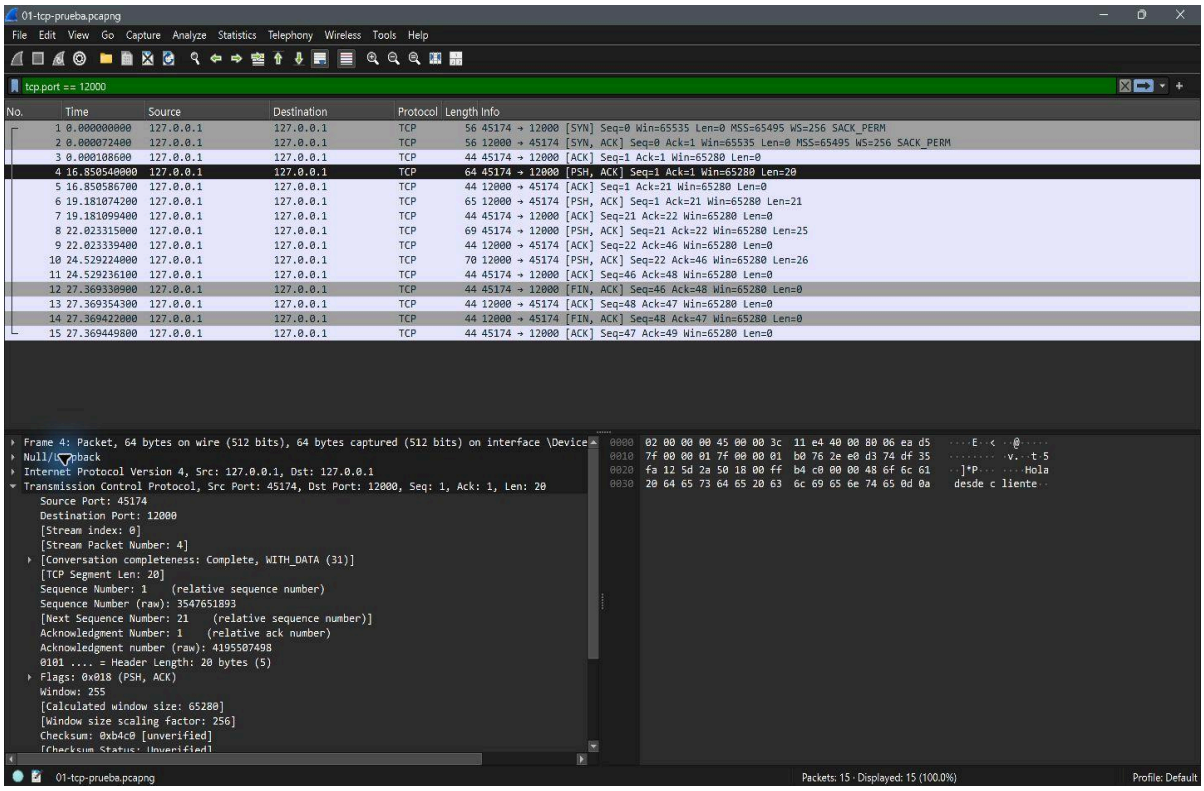
Experimento TCP. Iniciar la captura en loopback con el filtro tcp.port == 12000. Abrir dos terminales:

```
# Terminal A (servidor): escucha en el puerto TCP 12000
ncat -v -l 12000
```

```
# Terminal B (cliente): se conecta al servidor
ncat -v 127.0.0.1 12000
```

Escriban una línea en la terminal B y presionen Enter; luego respondan desde la A. Manden 2 o 3 mensajes en cada sentido. Finalmente cierren el cliente (Ctrl+C en B) y detengan la captura. Capturen el paquete con Wireshark, recuerden filtrar con tcp.port == 12000 así no se les hace una ensalada de paquetes en el sniffer.

Prueba en Windows con Ncat 7.993 y Wireshark 4.6.9, interfaz Npcap loopback. Servidor: ncat -v -l 127.0.0.1 12000. Cliente: ncat -v 127.0.0.1 12000. Enviamos dos líneas por sentido; la captura contiene 15 paquetes: handshake (1–3), datos y ACK (4–11), y cierre con Ctrl+C (12–15).



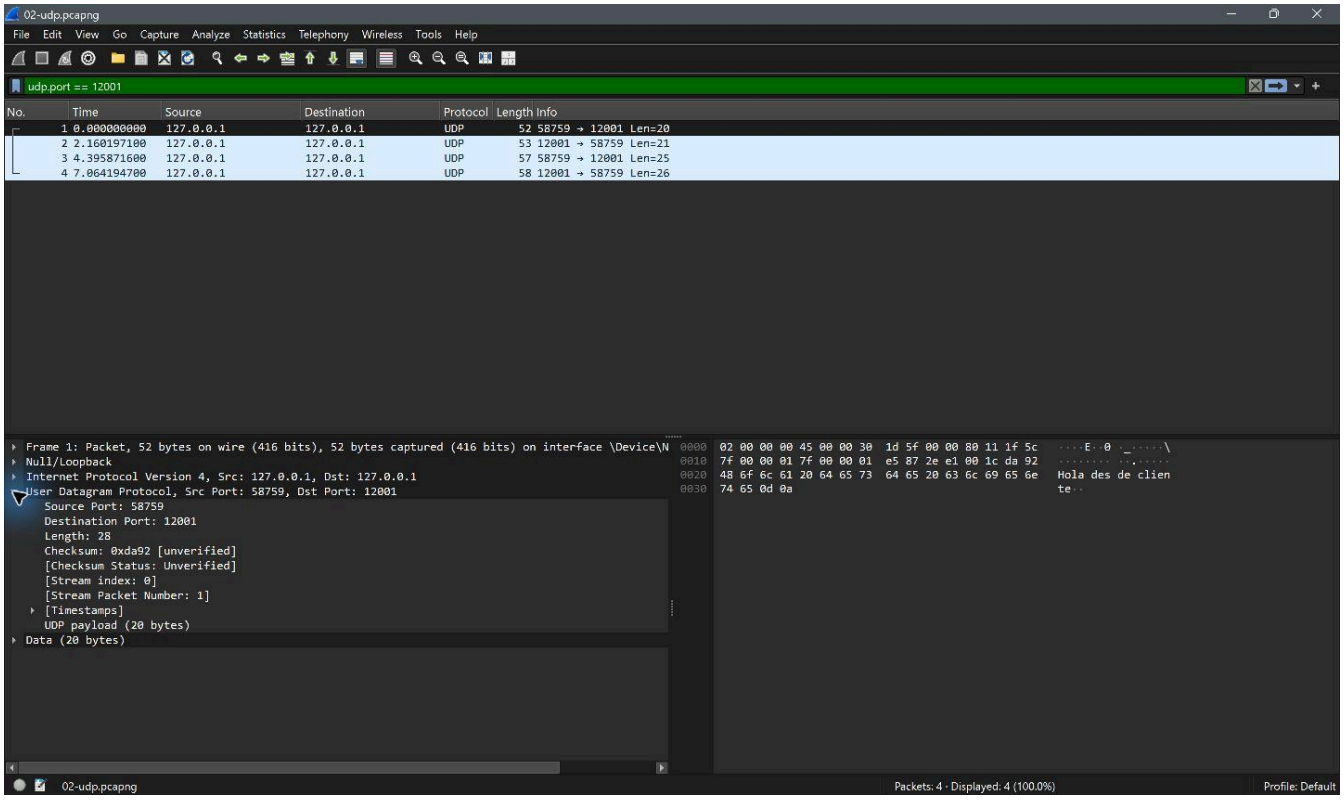
Experimento UDP. Nueva captura en loopback, ahora con el filtro udp.port == 12001:

```
# Terminal A (servidor)
ncat -v -u -l 12001
```

```
# Terminal B (cliente)
ncat -v -u 127.0.0.1
```

Envíen mensajes en ambos sentidos (el servidor puede responder una vez que recibió el primer datagrama del cliente) y cierren con Ctrl+C.

Servidor: ncat -v -u -l 127.0.0.1 12001. Cliente: ncat -v -u 127.0.0.1 12001 (agregamos el puerto que falta en el ejemplo). Enviamos dos líneas por sentido: 4 datagramas, sin apertura ni cierre. En Wireshark decodificamos el contenido del puerto 12001 como Data para evitar una interpretación automática como otro protocolo.



a) **¿Qué pasó en la red cuando ejecutaron el comando del cliente, antes de escribir el primer mensaje? Compárenlo con TCP.**

En UDP no apareció tráfico hasta escribir el primer mensaje: no hay handshake. En TCP aparecieron primero SYN, SYN-ACK y ACK (paquetes 1 a 3), antes de enviar datos. El mensaje “Connected” de ncat en UDP no confirma que exista un servidor.

b) ¿Cuántos datagramas generó cada mensaje? ¿Hay algo parecido a un ACK?

Cada línea corta generó un datagrama: cuatro mensajes, cuatro datagramas (dos por sentido). UDP no envió ACK ni retransmisiones. Las respuestas escritas por el servidor son datos de la aplicación.

c) Comparen el encabezado UDP con el encabezado TCP de un segmento con datos: ¿qué campos tiene cada uno? ¿Cuántos bytes ocupa cada encabezado?

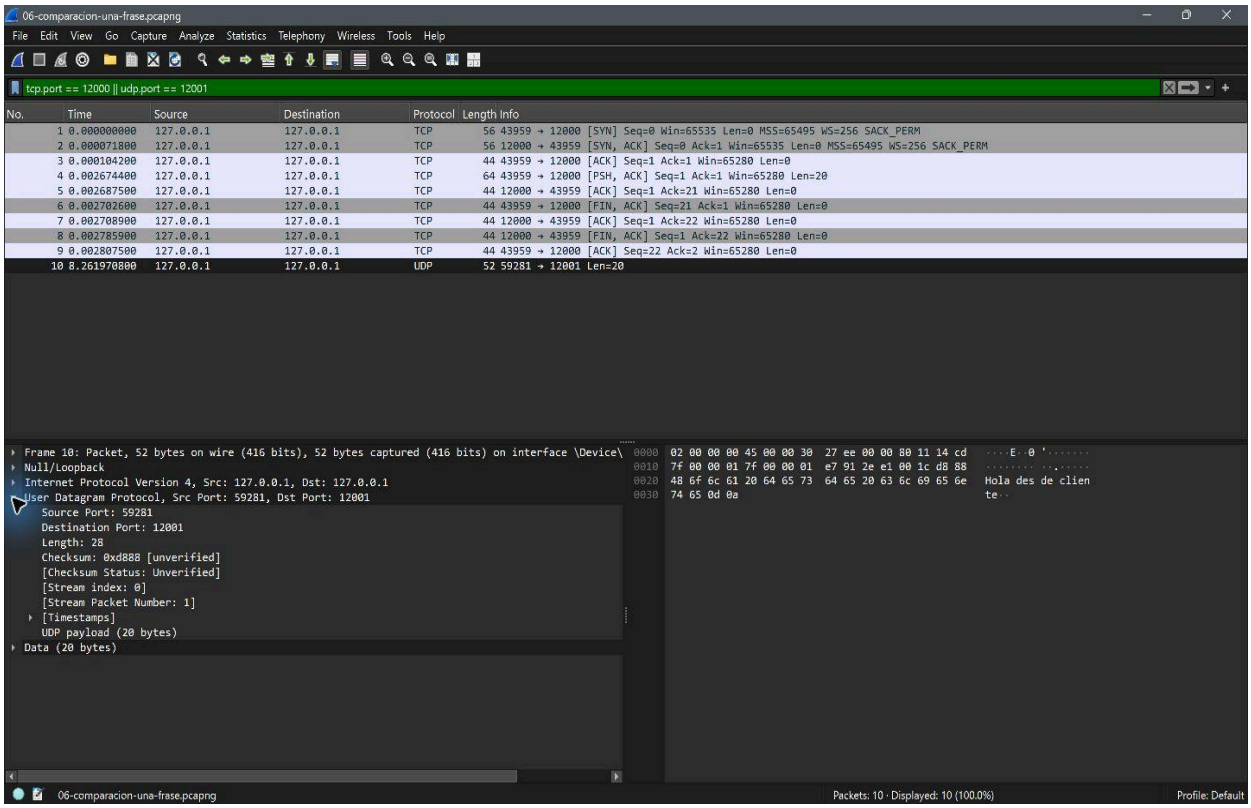
UDP tiene puertos origen/destino, longitud y checksum: 8 bytes. TCP agrega secuencia, ACK, longitud del encabezado, bits reservados, flags, ventana y puntero urgente, además de puertos y checksum. En el segmento de datos capturado ocupa 20 bytes; con opciones puede llegar a 60. Los SYN de esta prueba ocupan 32 bytes.

d) ¿Qué pasó en la red al cerrar el cliente con Ctrl+C? ¿Y en TCP?

En UDP, Ctrl+C cerró el proceso sin generar paquetes de cierre. En TCP hubo cuatro segmentos: FIN-ACK, ACK, FIN-ACK y ACK (12 a 15). Cada extremo cerró su sentido de envío.

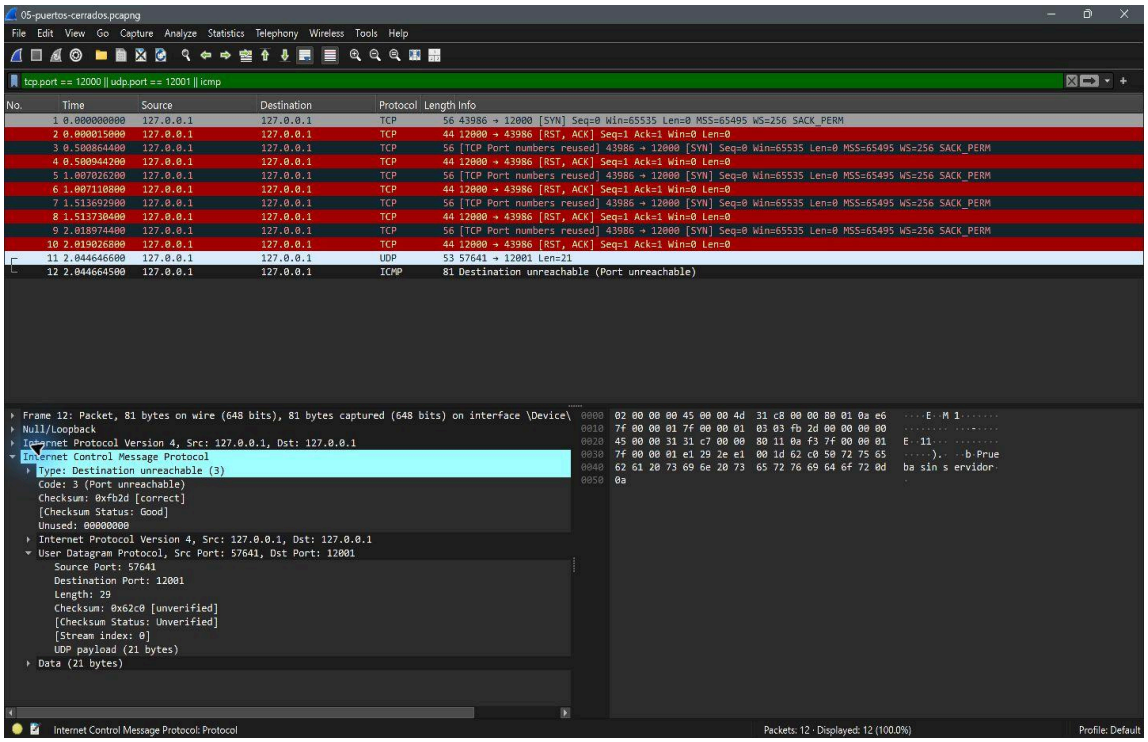
e) Para enviar la misma frase, ¿cuántos paquetes necesitaron en total con TCP y cuántos con UDP? ¿Qué "compran" con los paquetes extra de TCP?

En una prueba adicional enviamos “Hola desde cliente” (20 bytes con CRLF), usando --send-only y cierre al terminar la entrada: TCP necesitó 9 paquetes (3 de apertura, datos + ACK y 4 de cierre), UDP, 1. En la sesión de cuatro mensajes fueron 15 frente a 4. TCP aporta entrega ordenada, detección de pérdidas, retransmisiones y control de flujo/congestión. El total puede variar según ACK combinados y cierre.



f) ¿Y si nadie escucha? Con Wireshark capturando en loopback y el filtro tcp.port == 12000 || udp.port == 12001 || icmp, sin servidores corriendo:

TCP recibió RST-ACK ante cada SYN y ncat informó conexión rechazada. Se observaron cinco intentos. UDP envió un datagrama (11) y recibió ICMP Destination Unreachable, tipo 3, código 3: Port Unreachable (12). No se estableció una conexión TCP ni hubo un servidor UDP recibiendo el mensaje.




```
ncat -v 127.0.0.1 12000          # TCP a un puerto cerrado
ncat -v -u 127.0.0.1 12001       # UDP a un puerto cerrado, escribir y enter
```

4) Servidor TCP mínimo - TP5 Redes de Computadoras.

Un socket es el "punto final" de una comunicación que el sistema operativo le entrega a un programa: el programa escribe y lee bytes en el socket, y el sistema operativo se ocupa de TCP/UDP, IP y la placa de red. No vamos a entrar en tanta teoría aca, ya está bastante pesado el TP. A programar.

Ahora hacemos nosotros lo que hacía ncat. Los scripts están resueltos para que puedan concentrarse en relacionar cada línea con lo que ocurre en la red.

a) Leer el código del servidor

tcp_server.py > ...

```python
import socket
import sys

HOST = sys.argv[1] if len(sys.argv) > 1 else "127.0.0.1"
PORT = int(sys.argv[2]) if len(sys.argv) > 2 else 12000

# 1. Crear el socket. AF_INET = IPv4, SOCK_STREAM = TCP.
#    Esto es una operación local: todavía no viaja nada por la red.
servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Permite reiniciar el servidor enseguida sin el error "Address already in use"
# (ver TIME_WAIT en la guía). No es imprescindible para entender el resto.
servidor.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

# 2. Asociar el socket a una IP y un puerto locales (local, sin tráfico).
servidor.bind((HOST, PORT))

# 3. Pedirle al sistema operativo que acepte conexiones en ese puerto.
#    A partir de acá, un SYN que llegue a este puerto será respondido con SYN-ACK.
servidor.listen(1)
print(f"[servidor TCP] escuchando en {HOST}:{PORT} ...")

# 4. Esperar a que un cliente complete el three-way handshake.
#    accept() devuelve un socket NUEVO, dedicado a esta conexión,
#    y la dirección (IP, puerto) del cliente.
conexion, direccion_cliente = servidor.accept()
print(f"[servidor TCP] conexión aceptada desde {direccion_cliente[0]}:{direccion_cliente[1]}")

# 5. Recibir hasta 1024 bytes. TCP entrega un flujo de bytes: recv() puede
#    devolver menos (o más mensajes juntos) de lo que el cliente envió en un send().
datos = conexion.recv(1024)

# 6. Mostrar lo recibido (bytes -> texto).
mensaje = datos.decode("utf-8", errors="replace")
print(f"[servidor TCP] recibido ({len(datos)} bytes): {mensaje!r}")

# 7. Enviar una respuesta sencilla.
respuesta = f"Recibido: {mensaje}"
conexion.sendall(respuesta.encode("utf-8"))
print(f"[servidor TCP] respuesta enviada: {respuesta!r}")

# Esperar a que el cliente cierre: recv() devuelve b"" cuando llega su FIN.
while conexion.recv(1024):
    pass
print("[servidor TCP] el cliente cerró la conexión")

conexion.close()
servidor.close()
```

➔ El servidor crea un socket TCP con socket(), indicando IPv4 (AF_INET) y TCP

(SOCK_STREAM). Luego lo asocia a la dirección 127.0.0.1 y al puerto 12000 con bind(), y con listen() le indica al sistema operativo que acepte conexiones en ese puerto. Estas tres llamadas solo preparan el socket dentro del sistema operativo.

A continuación, el programa queda bloqueado en accept() hasta que un cliente se conecta. accept() devuelve un socket nuevo, dedicado a esa conexión, y la dirección del cliente. Con ese socket el servidor recibe el mensaje con recv(), lo convierte de bytes a texto y responde con sendall(). Finalmente espera a que el cliente cierre la conexión (recv() devuelve b"" cuando llega su FIN) y cierra con close().

b) Leer el código del cliente:

```
python tcp_client.py                          # se conecta a 127.0.0.1:12000
python tcp_client.py 192.168.43.20            # IP del servidor (Parte 7)
python tcp_client.py 192.168.43.20 "Hola!"    # IP y mensaje
```

tcp_client.py > ...

```python
import socket
import sys

SERVER_IP = sys.argv[1] if len(sys.argv) > 1 else "127.0.0.1"
MENSAJE = sys.argv[2] if len(sys.argv) > 2 else "Hola servidor"
SERVER_PORT = 12000

# 1. Crear el socket TCP (operación local).
cliente = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
cliente.settimeout(5)  # evita quedar bloqueado para siempre si el servidor no responde

# 2. Conectarse: acá el sistema operativo envía el SYN y completa el
#    three-way handshake. El puerto local del cliente lo elige el SO.
try:
    cliente.connect((SERVER_IP, SERVER_PORT))
except ConnectionRefusedError:
    sys.exit("[cliente TCP] conexión rechazada: el host respondió, pero nadie escucha en ese puer
except (socket.timeout, TimeoutError):
    sys.exit("[cliente TCP] sin respuesta al intento de conexión (¿firewall? ¿IP equivocada? ¿ais

ip_local, puerto_local = cliente.getsockname()
print(f"[cliente TCP] conectado a {SERVER_IP}:{SERVER_PORT} desde {ip_local}:{puerto_local}")

# 3. Enviar el mensaje (texto -> bytes).
cliente.sendall(MENSAJE.encode("utf-8"))
print(f"[cliente TCP] enviado: {MENSAJE!r}")

# 4. Recibir la respuesta.
datos = cliente.recv(1024)

# 5. Mostrarla.
print(f"[cliente TCP] respuesta ({len(datos)} bytes): {datos.decode('utf-8', errors='replace')!r}")
```

➔ El cliente crea un socket TCP con socket() y se conecta al servidor con connect(). No elige su propio puerto: el sistema operativo le asigna uno de forma automática. Luego envía el mensaje con sendall(), espera la respuesta con recv(), la muestra en pantalla y cierra la conexión con close().

La IP del servidor y el mensaje se pueden pasar como argumentos. Si no se indican, se usan 127.0.0.1 y un mensaje por defecto. Además, el código contempla dos fallas

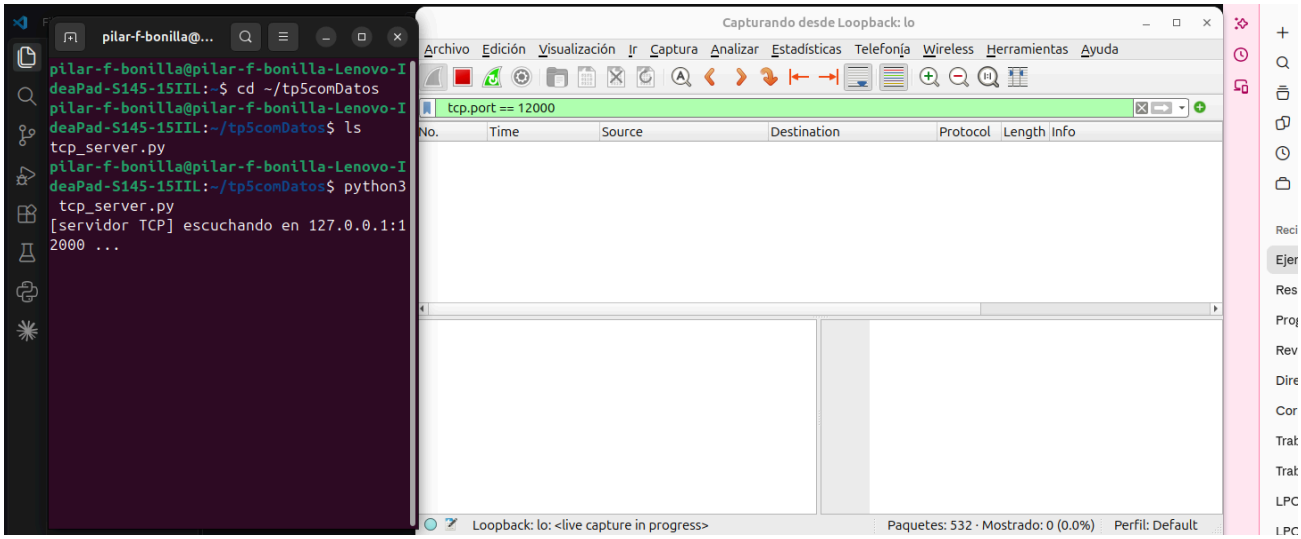
posibles en connect(): si el host responde pero nadie escucha en el puerto, se recibe un RST y la conexión es rechazada. Si no llega ninguna respuesta (por ejemplo, por una IP inexistente o un firewall), se agota el tiempo de espera de 5 segundos.

c) Ejecutar (capturando en loopback con el filtro tcp.port == 12000):

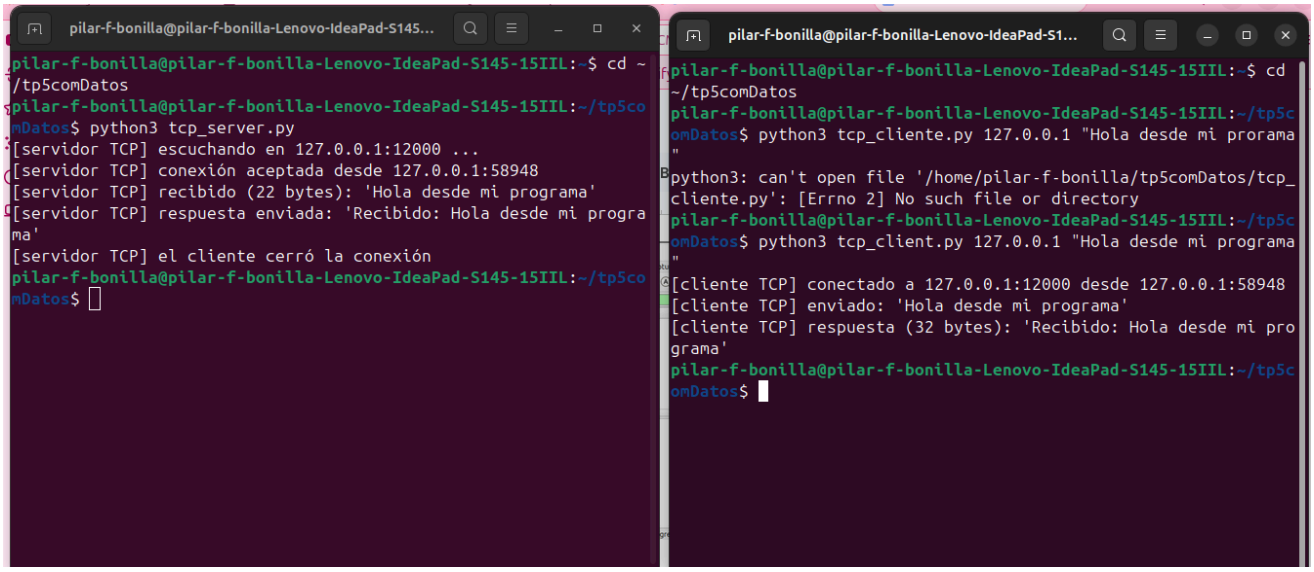
Comportamiento esperado (el puerto del cliente va a ser otro): [servidor TCP] escuchando en 127.0.0.1:12000 ...
[servidor TCP] conexión aceptada desde 127.0.0.1:48096 [servidor TCP] recibido (22 bytes): 'Hola desde mi programa'
[servidor TCP] respuesta enviada: 'Recibido: Hola desde mi programa' [servidor TCP] el cliente cerró la conexión

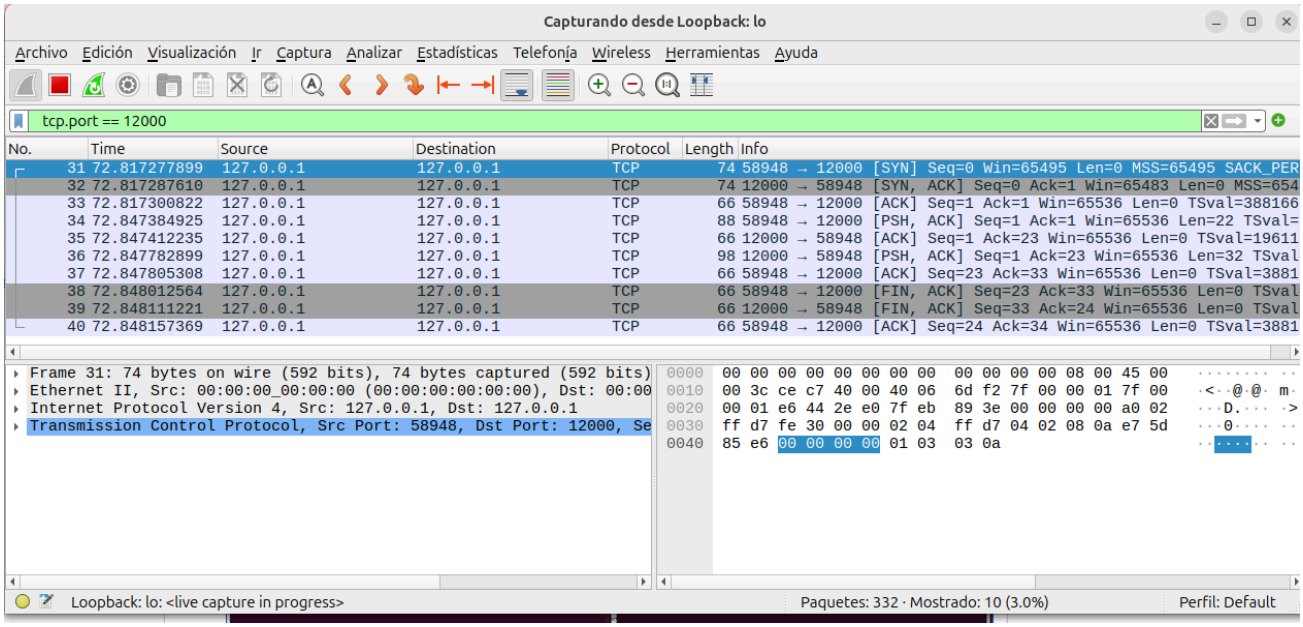
[cliente TCP] conectado a 127.0.0.1:12000 desde 127.0.0.1:48096 [cliente TCP] enviado: 'Hola desde mi programa'
[cliente TCP] respuesta (32 bytes): 'Recibido: Hola desde mi programa' El servidor atiende un cliente y termina. Para repetir, vuelvan a ejecutarlo.

Se observó en la terminal de linux:

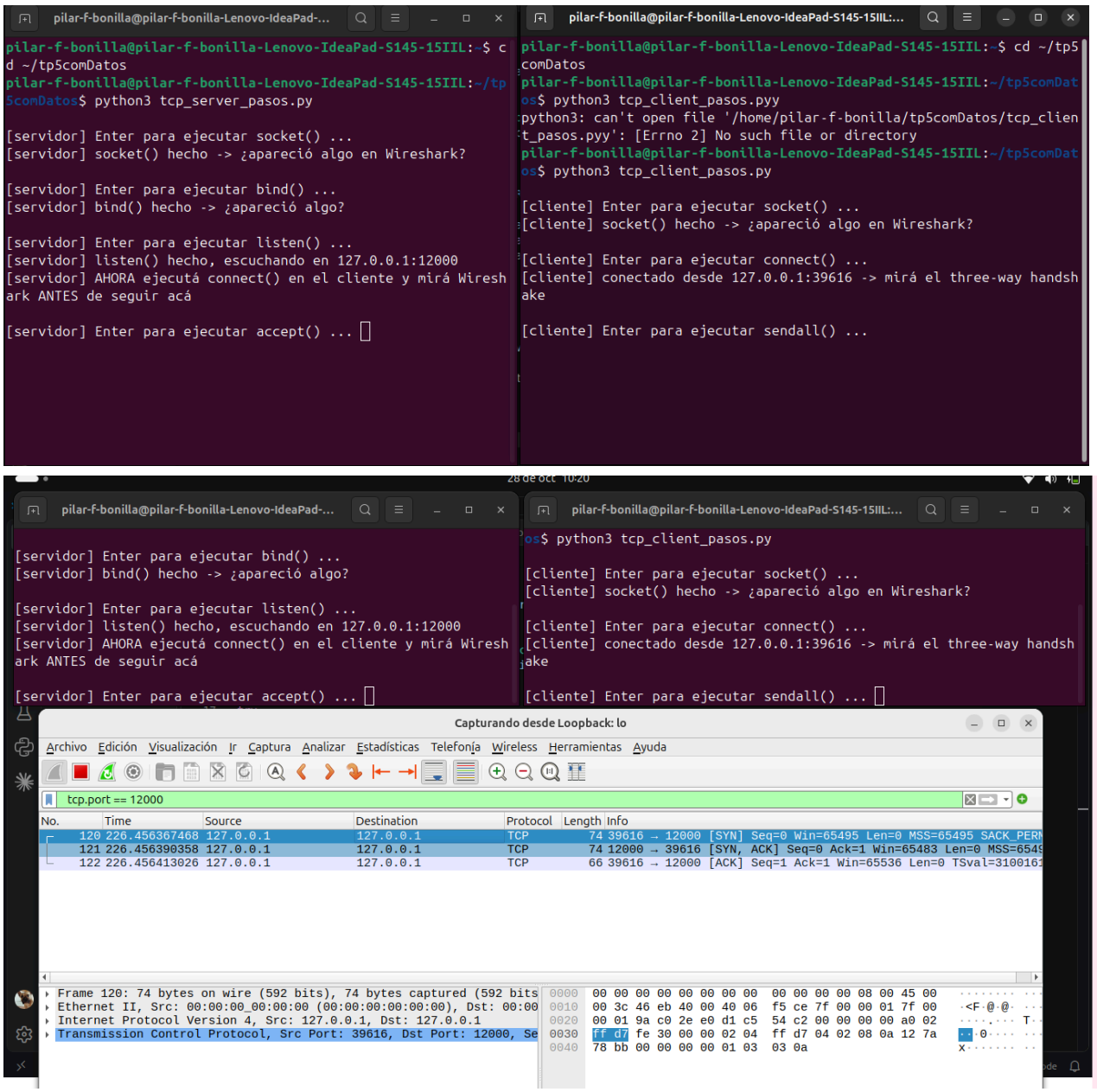


➔ Primero ejecutamos ambos programas de forma normal, capturando con Wireshark en la interfaz lo con el filtro tcp.port == 12000. En las siguientes capturas se observa la conexión completa: el three-way handshake, el intercambio del mensaje “Hola desde mi programa” (22 bytes) y su respuesta (32 bytes), y el cierre de la conexión.



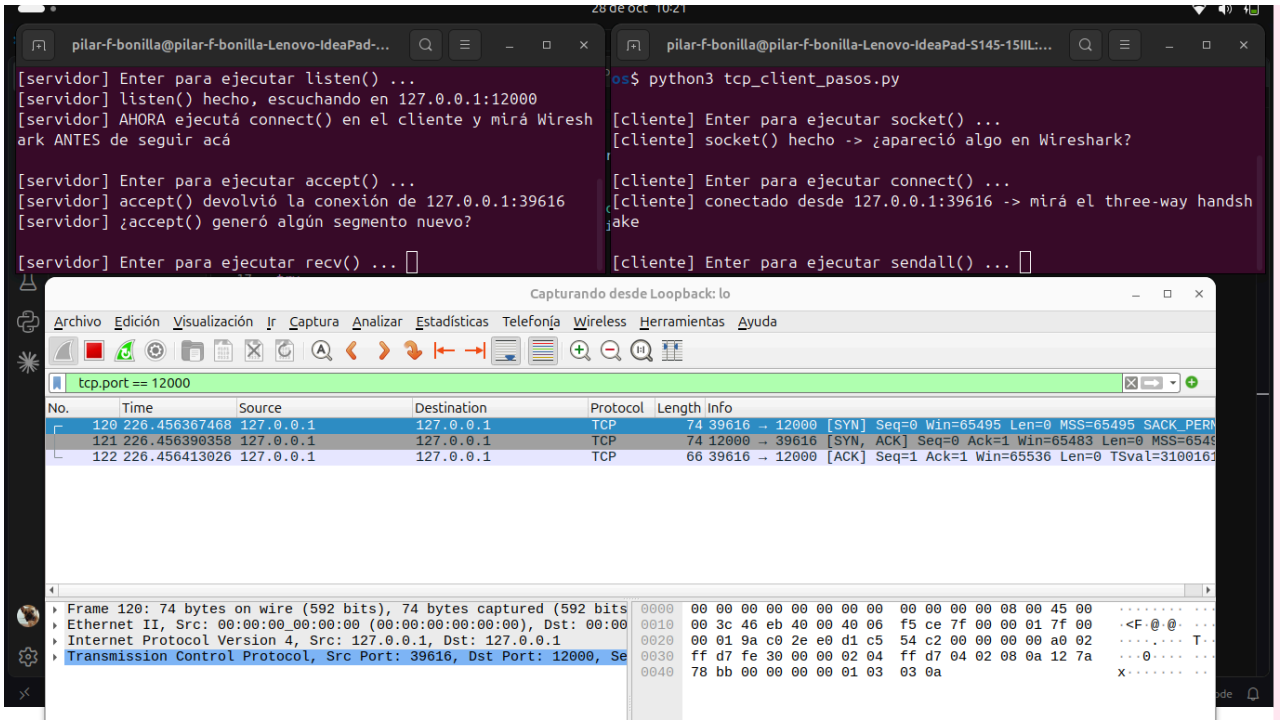


➔ En la siguiente parte, se repitió el proceso con una versión “paso a paso” de los scripts, que se detienen antes de cada llamada y espera a que se presione enter . De esta forma se observó en Wireshark el efectos de cada llamada por separado y verificar en particular que el handshake se completa antes de que el servidor ejecute accept().

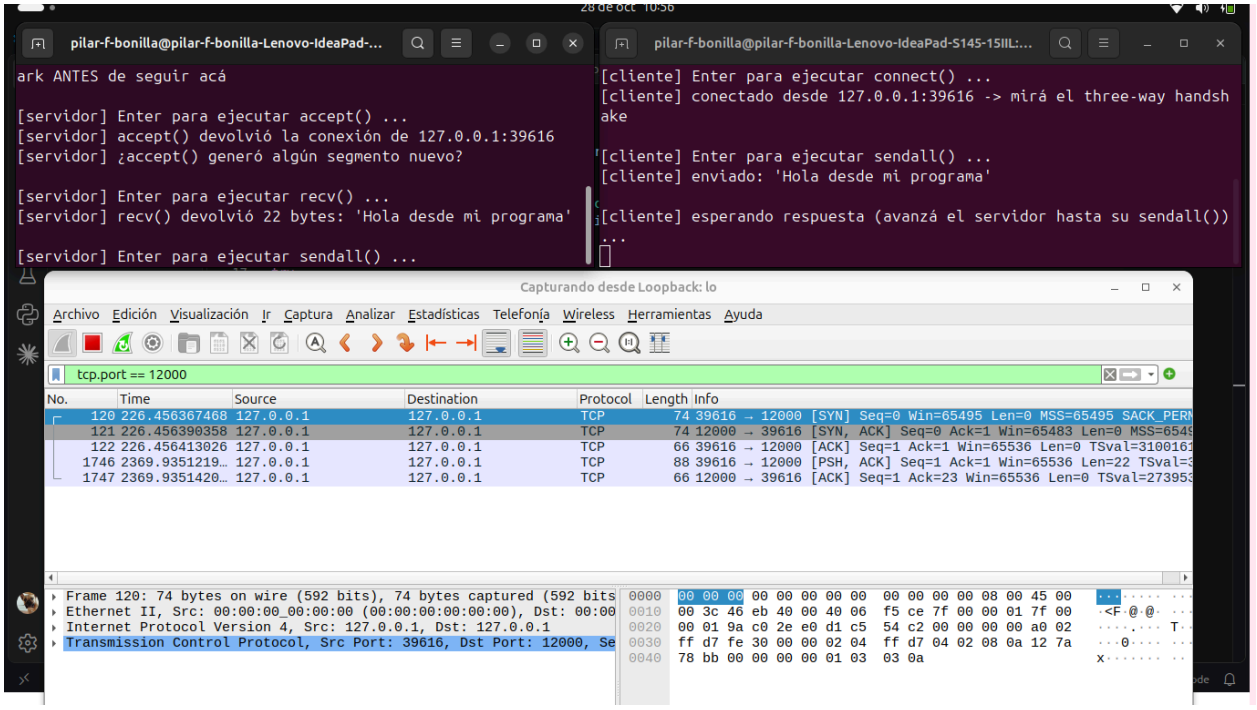


➔ El servidor ejecutó socket(), bind() y listen() y quedó detenido antes de accept(). Durante esos pasos Wireshark no registró ningún segmento en el puerto 12000. Esto confirma que son operaciones locales: solo crean el socket, le asignan la dirección 127.0.0.1:12000 y le indican al sistema operativo que acepte conexiones en ese puerto

Con el servidor detenido antes de accept(), el cliente ejecutó connect() y Wireshark registró el three-way handshake completo (paquetes 120 [SYN], 121 [SYN, ACK] y 122 [ACK]). El cliente ya consideraba la conexión establecida. Esto demuestra que el handshake lo realiza el kernel del servidor a partir del listen(), y que accept() no genera tráfico: solo entrega al programa una conexión que el sistema operativo ya había establecido.

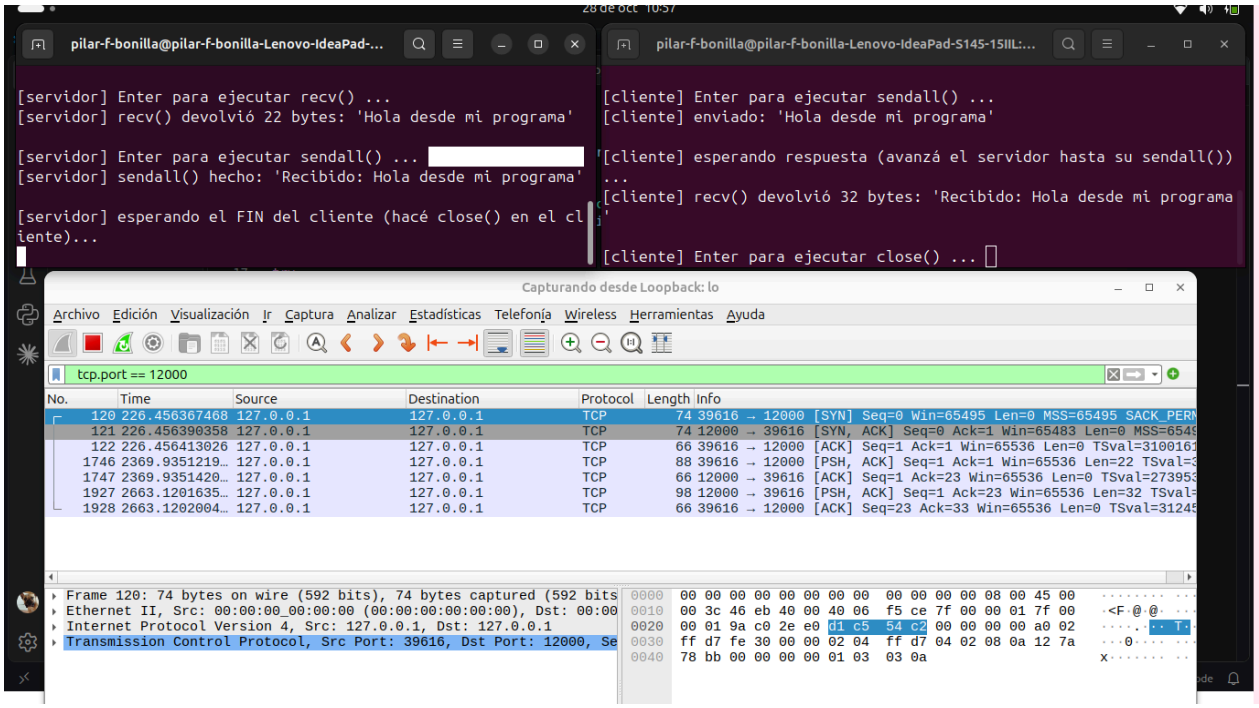


➔ El servidor ejecutó accept(), que devolvió la conexión desde 127.0.0.1:39616, y Wireshark no registró ningún segmento nuevo: accept() no genera tráfico.

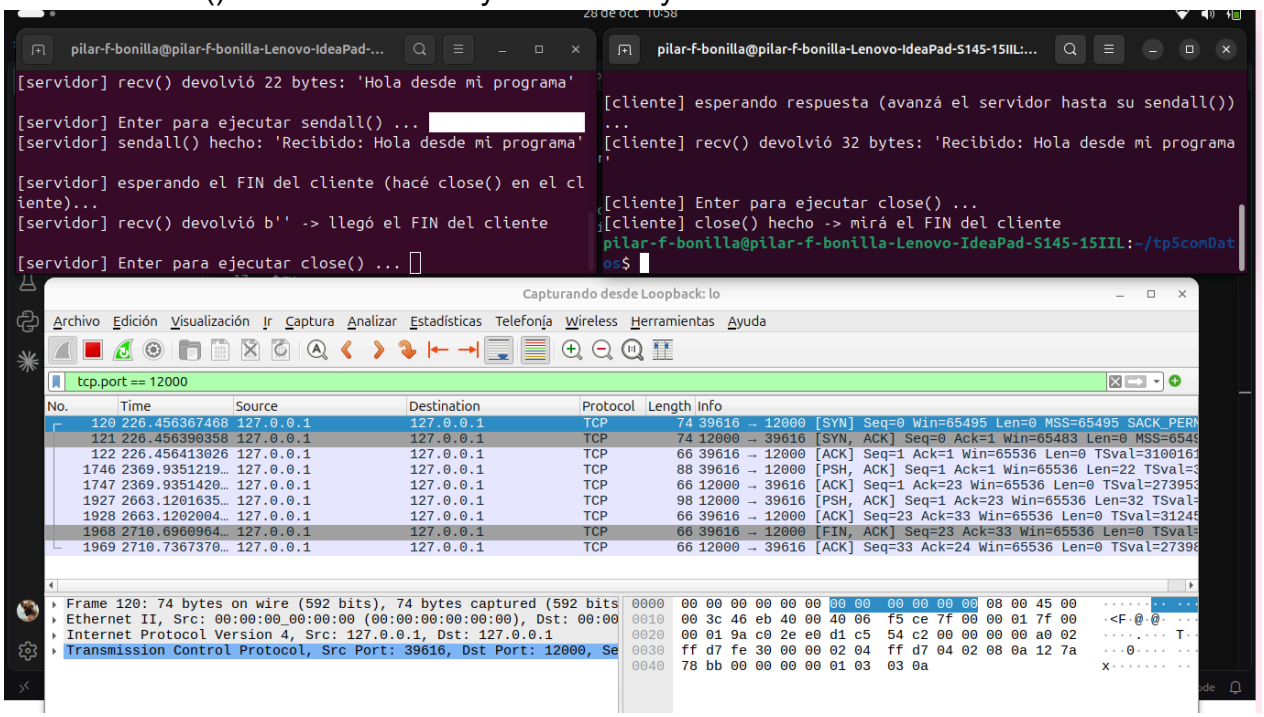


➔ Con el servidor detenido antes de recv(), el cliente ejecutó sendall(). Wireshark registró el segmento 1746 [PSH, ACK] con 22 bytes de datos y, de inmediato, el segmento 1747 [ACK] del servidor confirmándolos. El ACK se envió antes de que el programa llamara a recv(), lo que demuestra que lo genera el sistema operativo al recibir los datos y los guarda en un buffer.

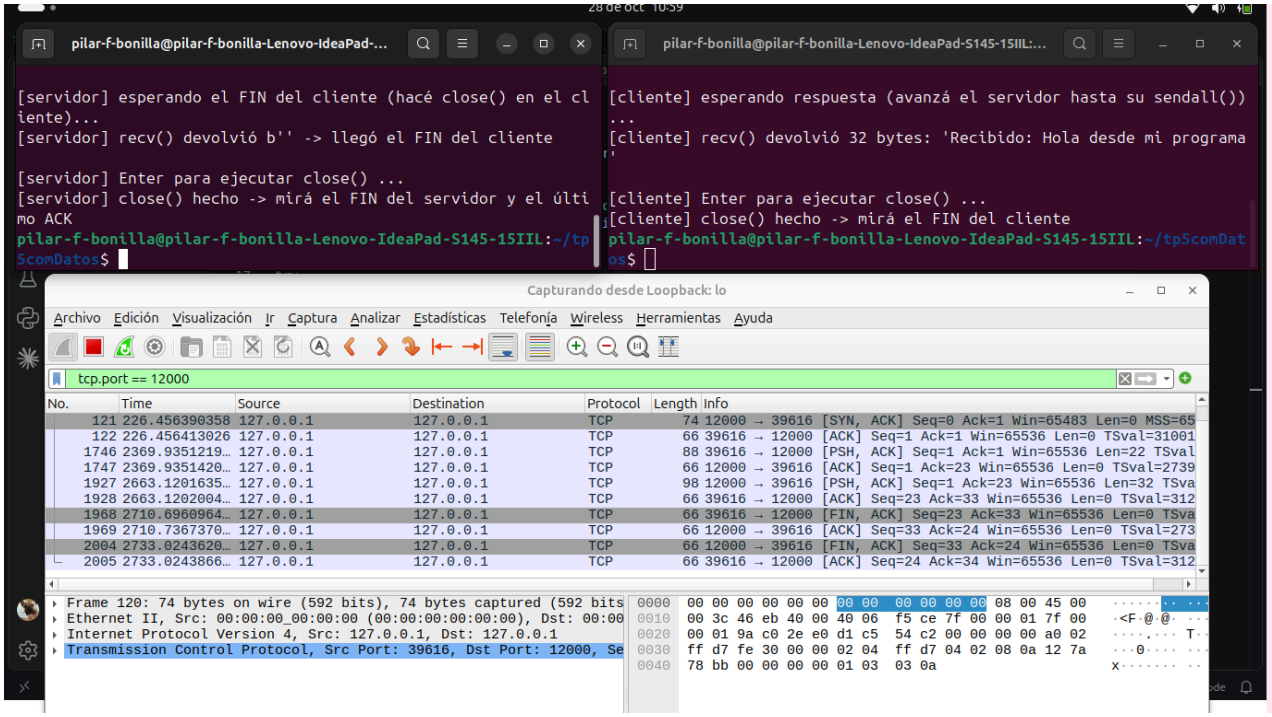
➔ El servidor ejecutó recv(), que devolvió los 22 bytes, y Wireshark no registró ningún segmento nuevo: recv() solo lee los datos que el sistema operativo ya tenía guardados en el buffer.



➔ El servidor ejecutó sendall() con la respuesta “Recibido: Hola desde mi programa”. Wireshark registró el segmento 1927 [PSH, ACK] con 32 bytes de datos y el segmento 1928 [ACK] del cliente, con Ack=33. Ese ACK también lo generó el sistema operativo del cliente: el recv() del cliente solo leyó los datos ya recibidos.



➔ El cliente ejecutó close() y se registró el segmento 1968 [FIN, ACK] del cliente y el 1969 [ACK] del servidor (Ack=24, ya que el FIN ocupa un número de secuencia). El servidor detectó el cierre porque recv() devolvió b"". En este punto la conexión queda medio cerrada: el cliente ya no envía datos, pero el servidor todavía podría hacerlo.



➔ Por último, el servidor ejecuta close() y se registra el segmento 2004 [FIN, ACK] del servidor y el 2005 [ACK] del cliente, que completan el cierre. En esta ejecución el cierre tomó cuatro segmentos, porque el servidor demoró su close(). En la ejecución normal (paquetes 38 a 40) fueron tres, porque el servidor cerró enseguida y envió el ACK y su FIN en un mismo segmento. Ambos casos son válidos: TCP cierra cada sentido de la conexión por separado.

d) Con la captura, completar una tabla como la siguiente. Para cada llamada, indicar si genera tráfico y, si lo genera, qué segmentos (número de paquete en Wireshark y flags):

Llamada	**¿Dónde se ejecuta?**	**¿Genera tráfico?**	**Segmentos que observan**
socket()	**servidor y cliente**	NO	Ninguno: crea el socket en el SO
bind()	**servidor**	NO	Ninguno: asocia 127.0.0.1:12000 localmente

listen()	**servidor**	NO	Ninguno(desde acá el kernel responde SYN)
connect()	**cliente**	SI	120 [SYN], 121 [SYN, ACK], 122 [ACK]: three-way handshake
accept()	**servidor**	NO	Ninguno: el handshake (120–122) ocurrió antes de llamarlo
sendall()	**servidor**	SI	1927 [PSH, ACK] Len=32 y su ACK 1928 (el del cliente: 1746 [PSH, ACK] Len=22 y ACK 1747)
recv()	**servidor**	NO	Lee del buffer los datos que ya habían llegado en 1746. El ACK 1747 envió el kernel antes de que se llamara a recv().
close()	**ambos**	SI	Cliente: 1968 [FIN, ACK] y ACK 1969. Servidor: 2004 [FIN, ACK] y ACK 2005

➔ De esta forma podemos concluir que de las ocho llamadas analizadas, sólo tres generan tráfico: connect() (three-way handshake), sendall() (segmentos con datos) y close() (segmentos FIN). Las demás son operaciones locales. Además, varios segmentos no corresponden a ninguna línea del programa: el SYN-ACK y todos los ACK los genera el sistema operativo por su cuenta. El programa solo escribe y lee bytes en el socket, y el sistema operativo se encarga del protocolo TCP.